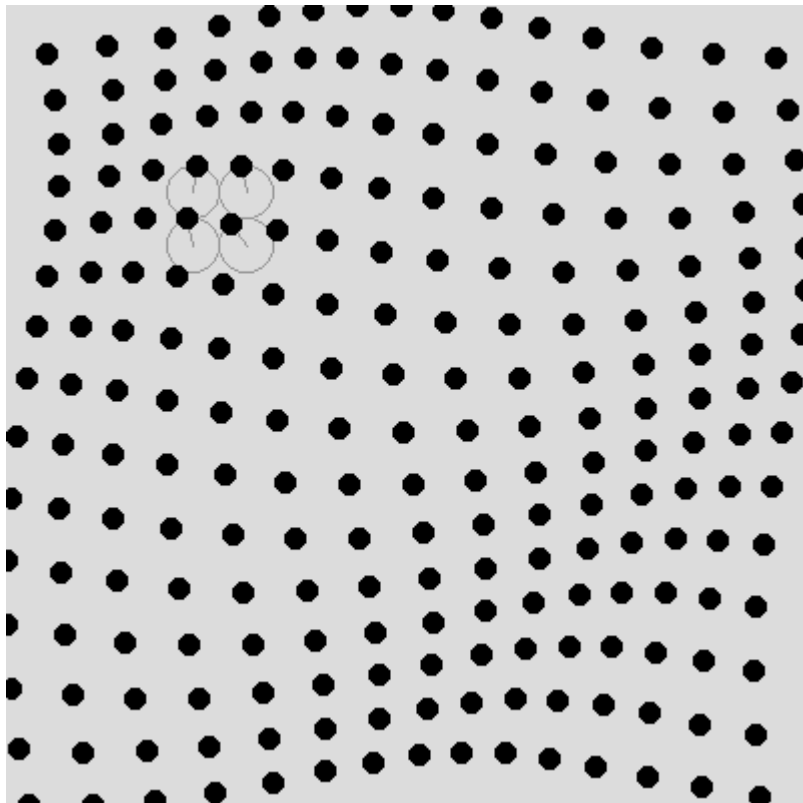


Cours 15 — Trigonométrie : une grille de points en rotation

Fichiers : ofApp15.h + ofApp15.cpp **Avant** : cours 13 (cos, sin, angles), cours 12 (grille par double boucle).

Une grille de 15 × 15 cases. Dans chaque case, un point tourne autour du centre de la case. Toutes les cases tournent à la même vitesse, mais chacune avec un **décalage** d'angle : l'ensemble ondule comme un drapeau.



■ 1. Le centre d'une case

```
tileSize = screenSize / (float)rows;
float x0 = i * tileSize + tileSize / 2;
float y0 = j * tileSize + tileSize / 2;
```

La case (i, j) commence en (i * tileSize, j * tileSize). Son centre est un demi-côté plus loin. C'est le calcul de position de grille du cours 12, avec + tileSize / 2 pour viser le milieu.

Le (float)rows mérite un mot. screenSize et rows sont deux int : 400 / 15 donnerait 26 (division entière, cours 01), et la grille ne remplirait pas la fenêtre. (float)rows **convertit** rows en float juste pour ce calcul : 26.67. Cette écriture (type)valeur s'appelle une conversion explicite ; on l'a vue dans l'autre sens au cours 14 avec (int)x.

■ 2. Un point qui tourne autour d'un centre

```
float x = radius * cos(angle) + x0;  
float y = radius * sin(angle) + y0;  
ofDrawCircle(x, y, 5);
```

La formule du cours 13, plus le centre : le point est à `radius` du centre `(x0, y0)`, dans la direction `angle`. Au cours 13 on déplaçait l'origine avec `ofTranslate` ; ici on ajoute le centre à la main, ce qui revient au même. Décommente les deux lignes de construction dans le code pour voir le cercle et le rayon de chaque case.

■ 3. Le déphasage

```
angle = angle + i * angleOffset + j * angleOffset;
```

Si toutes les cases avaient le même angle, les 225 points tourneraient en bloc, à l'unisson. En ajoutant un décalage qui dépend de la position de la case, les voisins sont légèrement en avance ou en retard l'une sur l'autre. Le résultat est une vague qui traverse la grille.

C'est la même idée que la traînée du cours 08, où la taille dépendait de `i` : **une grandeur commune à tous, plus un décalage propre à chacun**. Ici la grandeur commune est `globalAngle`, qui avance avec le temps ; le décalage propre est `i * angleOffset + j * angleOffset`.

Essaie `angleOffset = 0.1, 0.5, 1, 75`. À 75 radians par case, le décalage semble aléatoire alors qu'il est parfaitement régulier : le tour complet fait 6.28, et 75 tombe quelque part au hasard sur le cercle à chaque case.

■ 4. Une fonction par case

```
void ofApp::drawPoint(int i, int j, float angle) { ... }  
  
for (int i = 0; i < rows; i++) {  
    for (int j = 0; j < cols; j++) {  
        drawPoint(i, j, globalAngle);  
    }  
}
```

La double boucle parcourt les cases, la fonction dessine une case. Séparer « parcourir » et « dessiner un élément » rend les deux lisibles. Le corps de `drawPoint` ne sait rien de la grille : il reçoit `i`, `j` et l'angle, et fait son calcul.

■ 5. La vitesse angulaire

```
globalAngle = globalAngle + speed * ofGetLastFrameTime();
```

Le cours 06 avec un angle à la place d'une position : `speed` est en radians par seconde, multiplié par `dt` . 2.4 radians par seconde, c'est un peu plus d'un tour toutes les trois secondes.

■ Exercices

1. `angleOffset` à 0.3. Puis un déphasage qui ne dépend que de `i` : les colonnes ondulent ensemble.
2. Fais dépendre le rayon de la case : `radius * (i + 1) / rows` .
3. Remplace le point par un trait du centre au point : `ofDrawLine(x0, y0, x, y)` . Un champ de petites aiguilles.
4. Colore le point selon son angle avec `ofColor::fromHsb` : la teinte est un angle (cours 13).
5. Déphasage par la distance au centre de la fenêtre au lieu de `i + j` : des ondes circulaires.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp15.cpp` .

Le fichier `ofApp15.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```

#pragma once
#include "ofMain.h"

// 15 - Trigonométrie : grille de points en rotation
// Sketch d'origine : Cours 5/TrigoFlag
// Notions : grille par double boucle, calcul d'un centre de case, rotation locale
//           par cos / sin, déphasage selon la position (effet de vague / drapeau)

class ofApp : public ofBaseApp {
public:
    void setup();
    void update();
    void draw();

    void drawPoint(int i, int j, float angle);

    int    screenSize = 400;
    int    rows = 15;
    int    cols = 15;
    float tileSize = 0;    // calculé dans setup
    float radius = 0;
    float speed = 2.4f;    // radians par seconde (0.04 rad/frame x 60)
    float globalAngle = 0;
    float angleOffset = 75;
};

```

En tête du fichier `ofApp15.cpp`

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

```
#include "ofApp15.h"
```

Étape 1 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```

void ofApp::setup() {
    ofSetWindowShape(screenSize, screenSize);
    tileSize = screenSize / (float)rows;
    radius = tileSize / 2;
}

```

Étape 2 — `update()`

Exécuté à chaque frame, avant le dessin : tout ce qui change.

```
void ofApp::update() {  
    globalAngle = globalAngle + speed * ofGetLastFrameTime();  
}
```

Étape 3 — drawPoint()

Dessine un point qui tourne autour du centre de la case (i, j).

```
// Dessine un point qui tourne autour du centre de la case (i, j)  
void ofApp::drawPoint(int i, int j, float angle) {  
    // Déphasage : chaque case a un angle légèrement différent de ses voisines  
    angle = angle + i * angleOffset + j * angleOffset;  
  
    // Centre de la case  
    float x0 = i * tileSize + tileSize / 2;  
    float y0 = j * tileSize + tileSize / 2;  
  
    // Point sur le cercle de rayon "radius" autour de ce centre  
    float x = radius * cos(angle) + x0;  
    float y = radius * sin(angle) + y0;  
  
    // Décommenter pour visualiser la construction :  
    // ofNoFill(); ofSetColor(180); ofDrawCircle(x0, y0, radius);  
    // ofDrawLine(x0, y0, x, y);  
  
    ofFill();  
    ofSetColor(0);  
    ofDrawCircle(x, y, 5);  
}
```

Étape 4 — draw()

Exécuté à chaque frame, après update() : uniquement du dessin.

```

void ofApp::draw() {
    ofBackground(220);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            drawPoint(i, j, globalAngle);
        }
    }
}

```

■ Ce qu'il faut retenir

- Centre de la case (i, j) : $(i * \text{taille} + \text{taille} / 2, j * \text{taille} + \text{taille} / 2)$.
- $(\text{float})n$ convertit un entier en `float` pour éviter la division entière.
- Tourner autour d'un centre : $x = R * \cos(a) + x_0$, $y = R * \sin(a) + y_0$.
- Déphasage : une valeur commune plus un décalage propre à chaque élément.
- Une vitesse angulaire se traite comme une vitesse : radians par seconde, fois `dt`.